3.2 Velocity-based motion model

In this notebook we will explore **deterministic motion models** that, given a robot motion $\mathbf{u}$ and the previous robot pose $\mathbf{x}_{t-1}$, compute the new robot pose $\mathbf{x}_t$.

Depending on how the robot motion $\mathbf{u}$ is represented, two common formulations can be considered:

- **Velocity Motion Model:** the control input $\mathbf{u}_t$ is given by the robot velocities,

  $$\mathbf{u}_t = \begin{bmatrix} v_t \\ \omega_t \end{bmatrix},$$

  where v_t and ω_t denote the linear and angular velocities, respectively.
  Given the previous pose $\mathbf{x}_{t-1}$, the new pose is obtained through the velocity motion model:

  $$\mathbf{x}_t = g_v(\mathbf{x}_{t-1}, \mathbf{u}_t, \Delta t).$$

  Therefore, this formulation uses **two input components**: linear and angular velocity.

- **Odometry Motion Model:** instead of using velocities directly, the input is the **pose increment** estimated from the odometry readings between times $t-1$ and t,

  $$\Delta\mathbf{x}_t^{\text{odo}} = \begin{bmatrix} \Delta x_t \\ \Delta y_t \\ \Delta\theta_t \end{bmatrix}.$$

  This increment can be obtained from two consecutive odometry poses as

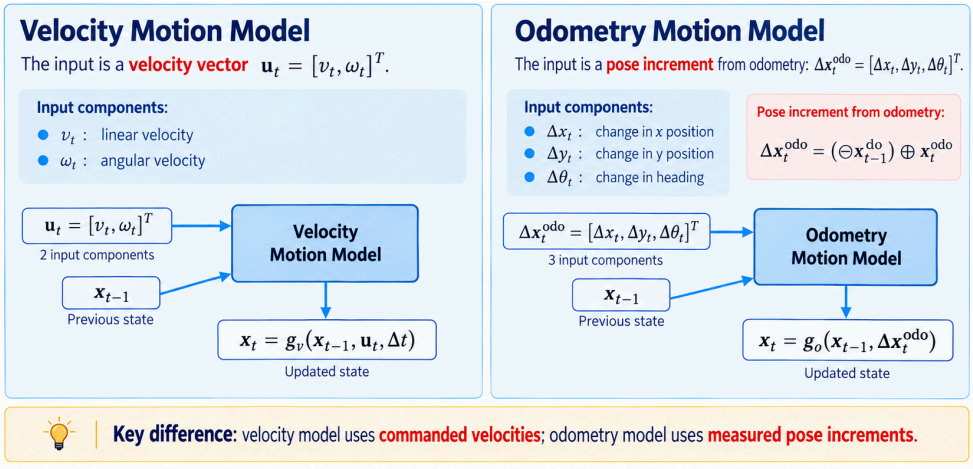
  $$\Delta\mathbf{x}_t^{\text{odo}} = (\ominus\mathbf{x}_{t-1}^{\text{odo}}) \oplus \mathbf{x}_t^{\text{odo}}.$$

  The robot pose is then updated according to

  $$\mathbf{x}_t = g_o(\mathbf{x}_{t-1}, \Delta\mathbf{x}_t^{\text{odo}}).$$

  In this case, the motion input contains **three components**: the increments in x, y, and orientation.

In summary:



Concretely, in this notebook we will focus on the **Velocity-based** motion model.

3.2.1 The model

> ℹ️ **Usage:** The *velocity motion model* is mainly used for motion planning, where the details of the robot's movement are of importance and odometry information is not available (*e.g.* no wheel encoders are available).

This motion model is characterized by the use of two velocities to control the robot's movement: **linear velocity** v and **angular velocity** ω. The velocity motion model defines the function $g_v(\cdot)$ as:

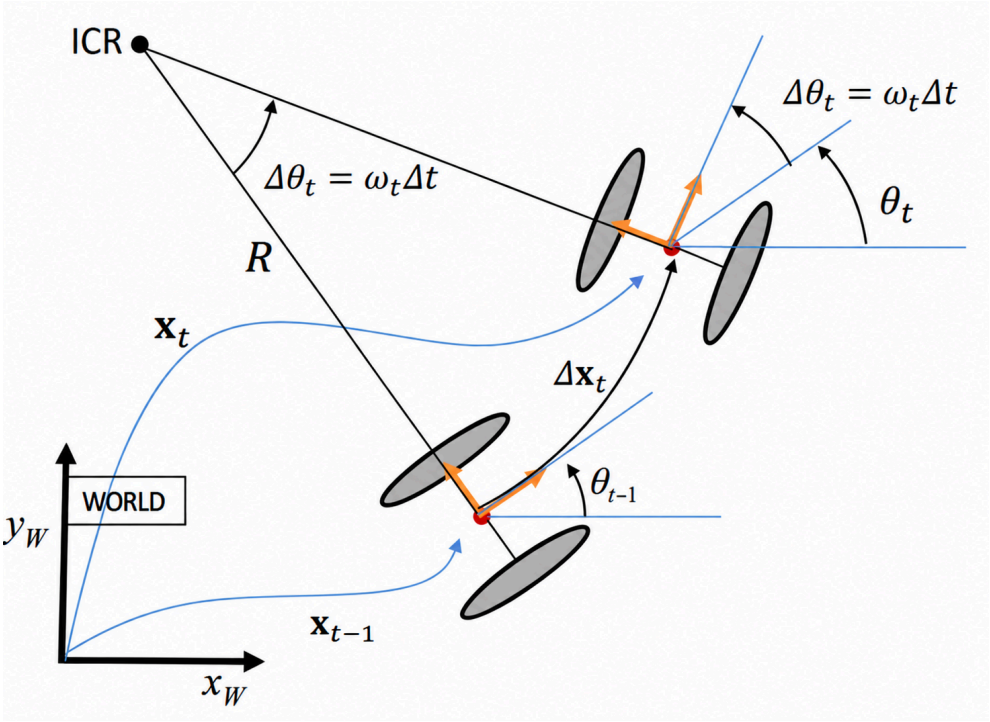
$$g_v(\mathbf{x}_{t-1}, \mathbf{u}_t, \Delta t) = \mathbf{x}_{t-1} \oplus \Delta \mathbf{x}_t(\mathbf{u}_t, \Delta t)$$

being $\Delta \mathbf{x}_t = [\Delta x_t, \Delta y_t, \Delta \theta_t]^T$ (assuming ω and v constant):

- $\Delta x_t = \frac{v_t}{\omega_t} \sin(\omega_t \Delta t)$
- $\Delta y_t = \frac{v_t}{\omega_t} [1 - \cos(\omega_t \Delta t)]$
- $\Delta \theta_t = \omega_t \Delta t$

Recall the curvature radius: $R = v_t / \omega_t$

The following figure illustrates this computation:



In this way, this motion model is characterized by the following equations, depending on the value of the angular velocity ω (note that a division by zero would appear in the first case if $\omega = 0$):

- If $\omega \neq 0$:

$$\begin{bmatrix} x_t \\ y_t \\ \theta_t \end{bmatrix} = \begin{bmatrix} x_{t-1} + \Delta x_t \cos\theta_{t-1} - \Delta y_t \sin\theta_{t-1} \\ y_{t-1} + \Delta x_t \sin\theta_{t-1} + \Delta y_t \cos\theta_{t-1} \\ \theta_{t-1} + \Delta\theta_t \end{bmatrix} = \begin{bmatrix} x_{t-1} \\ y_{t-1} \\ \theta_{t-1} \end{bmatrix} + \begin{bmatrix} -R\sin\theta_{t-1} + R\sin(\theta \\ R\cos\theta_{t-1} - R\cos(\theta_t \\ \Delta\theta_t \end{bmatrix}$$

- If $\omega = 0$:

$$\begin{bmatrix} x_t \\ y_t \\ \theta_t \end{bmatrix} = \begin{bmatrix} x_{t-1} \\ y_{t-1} \\ \theta_{t-1} \end{bmatrix} + v_t \cdot \Delta t \begin{bmatrix} \cos\theta_{t-1} \\ \sin\theta_{t-1} \\ 0 \end{bmatrix}$$

In [1]:
```
# In case of using Google Colab, mount Google Drive and enable interactive plots
try:
    from google.colab import drive, output
    drive.mount('/content/drive')
    # Change directory to this chapter's folder in your Drive copy of the course repo:
    %cd '/content/drive/MyDrive/Colab Notebooks/uma_robotics_2027/Chapter 03. Robot motion'
    %pip install -q ipympl
    # Make matplotlib detect the freshly installed ipympl backend:
    from matplotlib.backends.registry import backend_registry
    backend_registry.__init__()
    output.enable_custom_widget_manager()
```

```
except ImportError:
    pass

%matplotlib inline

# IMPORTS
import numpy as np
from numpy import random
import matplotlib.pyplot as plt
from IPython.display import display, clear_output
import time


import sys
sys.path.append("..")
sys.path.append("../..")
from utils.DrawRobot import DrawRobot
from utils.PlotEllipse import PlotEllipse
```

ASSIGNMENT 1: The model in action

Modify and properly call the following `next_pose()` function, used in the `VelocityRobot` class below, which computes the next pose $\mathbf{x}_t$ of a robot given:

- its previous pose $\mathbf{x}_{t-1}$,
- the velocity movement command $\mathbf{u}_t = [v_t, \omega_t]^T$, and
- a lapse of time Δt.

Concretely, you have to complete the if-else statement that takes into account whether the robot moves in a straight line (so $\omega_t = 0$). *Note: you don't have to modify the `None` in the function header nor in the `if cov is not None:` condition.*

Expected output:

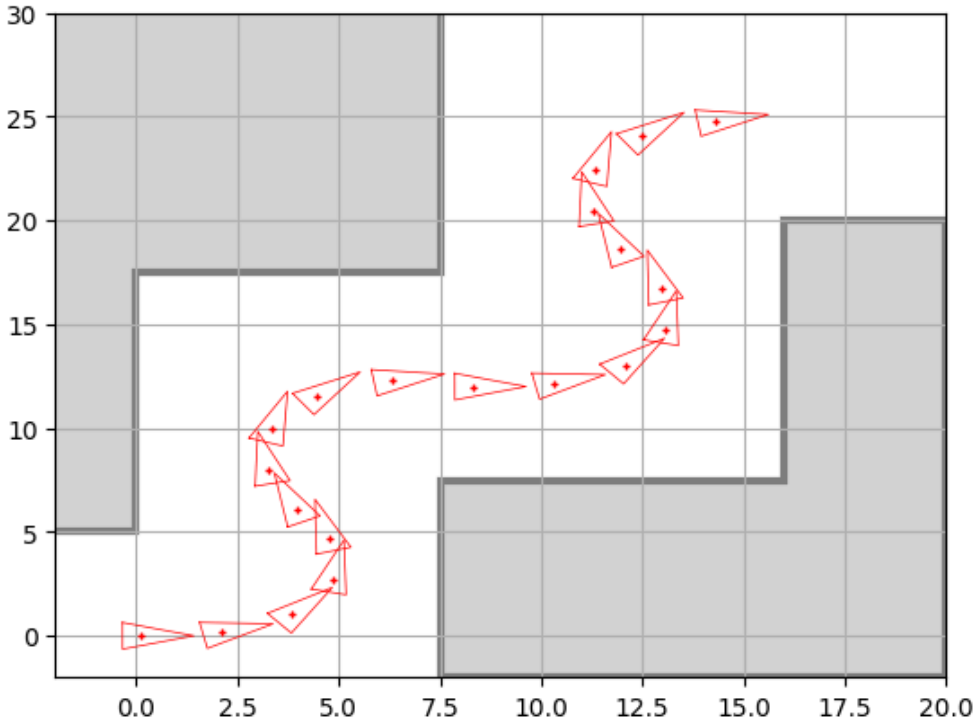


Fig. 2: Route of our robot.

In [2]:
```
def next_pose(x, u, dt, cov=None):
    ''' This function takes pose x and transforms it according to the
motion u=[v,w]'
        applying the differential drive model.

        Args:
            x: current pose
            u: differential command as a vector [v, w]'
            dt: Time interval in which the movement occurs
            cov: covariance of our movement. If not None, then add gaussian
noise
    '''
    if cov is not None:
        u += np.sqrt(cov) @ random.randn(2, 1)
        #u = np.random.multivariate_normal(u.flatten(),cov)

    if u[1] == 0: #linear motion w=0
        next_x = np.vstack([x[0] + u[0]*dt*np.cos(x[2]),
                            x[1] + u[0]*dt*np.sin(x[2]),
                            x[2]])
    else: #Non-linear motion w=!0
        R = u[0]/u[1] #v/w=r is the curvature radius
        next_x = np.vstack([x[0] - R*np.sin(x[2]) + R*np.sin(x[2] +
u[1]*dt),
                            x[1] + R*np.cos(x[2]) - R*np.cos(x[2] + u[1]*dt),
                            x[2] + u[1]*dt])

    return next_x
```

In [3]:
```
class VelocityRobot(object):
    """ Mobile robot implementation that uses velocity commands.
```

```
        Attr:
            pose: expected pose of the robot in the real world (without
taking account noise)
            dt: Duration of each step in seconds
    """
    def __init__(self, pose, dt):
        self.pose = pose
        self.dt = dt

    def step(self, u):
        self.pose = next_pose(self.pose, u, self.dt)

    def draw(self, fig, ax):
        DrawRobot(fig, ax, self.pose)
```

Test the movement of your robot using the demo below.

In [4]:
```
def main(robot, nSteps):

    v = 1 # Linear Velocity
    l = 0.5 #Half the width of the robot

    # MATPLOTLIB
    fig, ax = plt.subplots()
    plt.ion()
    fig.canvas.draw()
    plt.xlim((-2, 20))
    plt.ylim((-2, 30))
    plt.fill([7.5, 7.5, 16, 16, 20, 20],[-2, 7.5, 7.5, 20, 20, -2],
             facecolor='lightgray', edgecolor='gray', linewidth=3)
    plt.fill([-3, 0, 0, 7.5, 7.5, -3],[5, 5, 17.5, 17.5, 32, 32],
             facecolor='lightgray', edgecolor='gray', linewidth=3)

    plt.grid()

    # MAIN LOOP
    for k in range(1, nSteps + 1):
        #control is a wiggle with constant linear velocity
        u = np.vstack((v, np.pi / 10 * np.sin(4 * np.pi * k/nSteps)))

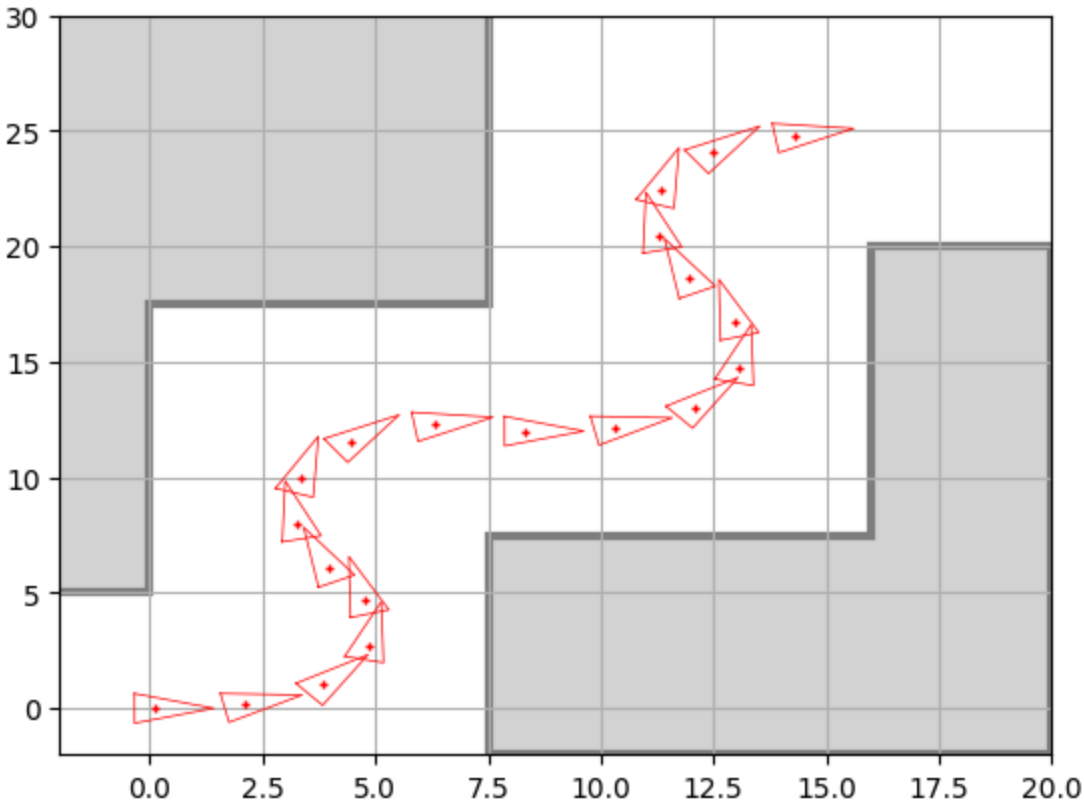
        robot.step(u)

        #draw occasionally
        if (k-1)%20 == 0:
            robot.draw(fig, ax)
            clear_output(wait=True)
            display(fig)
            time.sleep(0.1)

    plt.close()
```

In [5]:
```
# RUN
dT = 0.1 # time steps size
pose = np.vstack([0., 0., 0.])
```

```
robot = VelocityRobot(pose, dT)
main(robot, nSteps=400)
```



Thinking about it (1)

Now that you have some experience with robot motion and the velocity motion model, **answer the following questions**:

- Why do we need to consider two different cases when applying the $g_v(\cdot)$ function, that is, calculating the new robot pose?

  Porque la expresión general depende del radio de curvatura R = v/ω, que no está definido cuando ω = 0: aparecería una división por cero (en Python, `inf` o `nan` y una pose inválida). Geométricamente, si ω = 0 el robot no gira alrededor de ningún ICR a distancia finita (R → ∞) y el arco se convierte en una recta, así que se usa la expresión lineal $x_t = x_{t-1} + v\Delta t \cdot [\cos\theta, \sin\theta, 0]^T$. Esa expresión es el límite de la general cuando ω → 0, ya que $(v/\omega)\cdot\sin(\omega\Delta t) \to v\Delta t$ y $(v/\omega)\cdot[1 - \cos(\omega\Delta t)] \to 0$, de modo que ambos casos son coherentes entre sí y el modelo es continuo.

- How many parameters compound the motion command $\mathbf{u}_t$ in this model?

  Dos: la velocidad lineal v_t y la velocidad angular ω_t, es decir, $\boldsymbol{u}_t = [v_t, \omega_t]^T$, que se suponen constantes durante el intervalo Δt. El intervalo Δt no forma parte de la orden, sino que es un argumento más de $g_V(\cdot)$. En cambio, el modelo de odometría

usa tres componentes (Δx, Δy, Δθ). Con solo dos parámetros el robot diferencial no puede moverse lateralmente: siempre recorre arcos de circunferencia (o rectas), lo que refleja su restricción no holónoma.

AI Appendix

A.1 How I used AI

- **Model/tool:**

  Un asistente de programación basado en IA generativa (Devin con Claude Opus 5.5)

- **What I asked AI to help with (bullets):**
  - *Ayuda para completar `next_pose()` y `VelocityRobot.step()` y comprobar que la trayectoria coincide con la figura esperada.*
  - *Explicaciones del modelo de movimiento basado en velocidad (radio de curvatura, caso ω = 0).*
- **Best prompt I used (paste):**

  «Resuelve la práctica 3.2»

- **What I kept vs. changed from the AI output:**

  He revisado las ecuaciones del código contra las de la teoría y he comprobado la trayectoria obtenida antes de quedarme con la solución.

A.2 AI review

Ask one generative AI to review your memo using this prompt:

```
Act as a technical reviewer of this robotics notebook. Read
my memo and return exactly 5 bullet points:

- two on theory (key ideas),
- two on practice (relevant programming concepts of python
used, not particular functions or variables),
- one on how to improve the code (most critical aspect:
limitations, parameter choices, runtime/robustness, possible
bugs).

Keep each bullet ≤15 words and make them specific to my work.
```

Paste its **5** items here:

AI used for the review: an AI coding assistant.

- **(Theory 1)** AI claim:

  Con v y ω constantes, el robot recorre un arco de radio R = v/ω.

- **(Theory 2)** AI claim:

  El caso ω = 0 es el límite rectilíneo del arco; evita dividir por cero.

- **(Practice 1)** AI claim:

  Encapsulación: `VelocityRobot` *guarda pose y Δt y delega el cálculo en una función pura.*

- **(Practice 2)** AI claim:

  Vectorización NumPy: poses columna `(3, 1)` *construidas con* `vstack` *y funciones trigonométricas elementales.*

- **(Improvement)** AI claim:

  Comparar `ω == 0` *exacto es frágil; usar un umbral y normalizar θ.*

Finally, **do you agree with them?**, **would you have selected others?**

Estoy de acuerdo con los cinco puntos. Los teóricos recogen la idea central del modelo: con v y ω constantes la trayectoria de cada paso es un arco, y la recta es su caso límite, por lo que separar ω = 0 no es un parche sino una necesidad numérica. La mejora también me parece la más relevante: con ω muy pequeño pero no nulo (por ejemplo, tras añadir ruido), R se hace enorme y la diferencia de senos pierde precisión, así que convendría usar un umbral |ω| < ε y normalizar θ a (−π, π] para que no crezca indefinidamente. Como alternativa habría señalado que, si se pasa `cov`*,* `u += ...` *modifica in situ el array del llamador, lo que es un efecto secundario poco evidente. En la parte práctica habría destacado también la diferencia entre el modelo determinista implementado aquí y su versión con ruido, que el propio código ya deja preparada.*